SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

Assignment 2 – CO2

Operators, Decision-Making and Looping Constructs

Name	Prithiga S
Register No.	1965260110
Course Code	CSA0232
Course	C Programming
GitHub link	https://github.com/prithiga120808-rgb/CSA0232

Predict → Design → Implement → Measure → Analyse → Evaluate → Diagnose → Redesign → Validate

Q1 – Predict: Precedence & Associativity

6 Marks · K4

Predicted before running vs observed vs what the pharmacist intended

Bill (price, qty, senior)	Working (C rules)	Predicted total / net	Observed	Intended (5% GST on discounted)
B1: 100, 2, 1	200 − 30 + 10	180 / 180	180 / 180	**178.50 → wrong**
B2: 250, 4, 0	1000 − 0 + 50	1050 / 1000	1050 / 1000	**1050 / 1000 → same**
B3: 333, 3, 1	999 − 149 + 49	899 / 899	899 / 899	**891.61 → wrong**

Order in which operators are applied

- **total:** price*qty, then *15, then /100, then *senior (equal precedence, left-to-right).
- price*qty*5/100 is a separate term; then A − B, then + C (left-to-right).
- = is applied last (right-to-left).
- **net:** total > 1000 first, then ?:, then =.

Expressions that differ from intent

- **GST base:** 5% is charged on the gross price, not on the discounted price.
- **Integer division:** B3: 999*15/100 = 149 (not 149.85) and 999*5/100 = 49 (not 49.95); fractions of a rupee are lost.
- **Bill 2:** senior = 0, so the output is accidentally right.
- **Third statement:** count = count++ + 1 – undefined (next slide).

Q1 – Inquiry, Design & Alternative

6 Marks · K4

Why count = count++ + 1 is undefined behaviour

- count is modified twice (the ++ and the =) with no sequence point between them.
- C does not say which side effect happens first, so any result (even a crash) is allowed. Observed here: count = 6.

```
gcc -Wall q1.c
warning: operation on 'count' may be
undefined [-Wsequence-point]
```

```
double gross    = price * qty;
double discount = senior ? gross * 0.15 : 0;
double taxable  = gross - discount;
double gst      = taxable * 0.05;
double total    = taxable + gst;
double net      = total > 1000
                  ? total - 50 : total;
count++;   /* one clear statement */
```

Design decision – brackets AND intermediate variables

- Use double, not int, so there is no truncation.
- Name each step so the code reads like the bill, and bracket only where intent is not obvious.

Alternative: several statements, Big-O

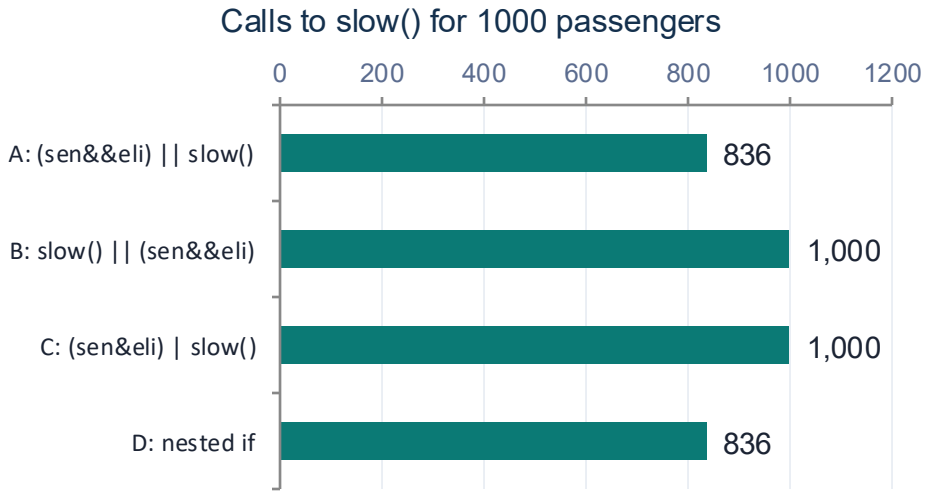
Measure	One expression	Intermediate vars
Time	~7 arithmetic ops, O(1)	same ops, O(1)
Space	O(1)	O(1): 3 extra doubles
Correctness	GST bug hidden	each step testable
Readability	poor	high

Q2 – Short-Circuit Evaluation

4 Marks · K4

Experiments (srand(42); 1000 passengers: 315 senior, 501 eligible class, 108 card holders)

```
(0 && (a = 1))  -> a stays 0    && skips 2nd operand when 1st is false
(1 || (b = 1))  -> b stays 0    || skips 2nd operand when 1st is true
(0 &  (a = 1))  -> a becomes 1  & ALWAYS evaluates both
4 && 2 = 1 (logical)     4 & 2 = 0 (bitwise: 100 & 010)
```



When does the order matter?

- **Matters:** when one operand is expensive. Put the cheapest / most-likely-false condition first in &&, and the most-likely-true first in ||.
- **Result:** A calls slow() only for the 1000 − 164 = 836 passengers not already decided; B calls it 1000 times.
- **Irrelevant:** when both are cheap and have no side effects.

Q2 – Changed Condition & Alternatives

4 Marks · K4

Changed condition: slow() writes an audit log

- Every passenger must now be logged, so slow() must run exactly n = 1000 times, in every order.
- Short-circuit would silently skip 164 audit entries (a correctness bug, not a speed-up).
- Recommended: call it first, store the result, then decide.

```
int card_ok = slow(card);   /* always logged */
if ((sen && eli) || card_ok) grant();
```

Alternative concept – calls to slow() for n passengers

Version	Calls (time)	Note
&& / || (short-circuit)	n – k (k = decided by cheap part)	836 observed
Nested if	n – k, same	more code, same cost
& and |	always n (O(n))	1000 observed
Call first (audit)	exactly n	required for the log

Space is O(1) in every version. Worst case is O(n) for all, but the short-circuit and nested-if versions do fewer calls whenever the cheap part decides first.

Evidence: raw counts come from q2.c (compiled with gcc -Wall, no warnings).

Q3 – Device Register: Complete Code

6 Marks · K4

```
unsigned char switch_on(unsigned char reg, int dev)
{ return reg | (1u << dev); }          /* TODO 1: set bit   */

unsigned char switch_off(unsigned char reg, int dev)
{ return reg & ~(1u << dev); }         /* TODO 2: clear bit */

unsigned char toggle(unsigned char reg, int dev)
{ return reg ^ (1u << dev); }          /* TODO 3: flip bit  */

int is_on(unsigned char reg, int dev)
{ return (reg >> dev) & 1u; }          /* TODO 4: test bit  */
```

Predicted vs observed register

Statement	Predicted	Observed
switch_on LIGHT	00000001	00000001
switch_on AC	00000101	00000101
toggle FAN	00000111	00000111
switch_off LIGHT	00000110	00000110

```
Output:
00000101
00000110
AC is ON, PUMP is OFF
```

Extra operations (bitwise only)

```
count ON : while (reg) { reg &= reg - 1; c++; }   /* devices ON = 2 */
```

Compile: gcc -Wall -o q3 q3.c Output matches the expected output exactly.

Q3 – Inquiry & Alternative

6 Marks · K4

Precedence: reg & 1 << dev vs (reg & 1) << dev

- << binds tighter than &, so reg & 1 << dev means reg & (1 << dev): it tests bit dev.
- (reg & 1) << dev tests only bit 0, then shifts the answer to position dev.
- Observed with reg = 0x04, dev = 2: first = 4 (bit is set), second = 0 (wrong).

Alternative: char state[8] (one byte per device)

Measure	1 byte, bit per device	char state[8]
set / clear / toggle / test	O(1)	O(1)
Count devices ON	O(8) = O(1) (O(k) with reg &= reg−1)	O(8) = O(1) loop
Space per home	1 byte	8 bytes
1,000,000 homes	1 MB	8 MB

signed char with bit 7 set

- The value is negative; promotion to int sign-extends it, so >> may fill with 1s (implementation-defined).
- Observed: signed 0x80 >> 7 = −1, unsigned = 1. Hence unsigned char.

Bits save 7× the memory with the same O(1) time. The array is simpler to read but wasteful on a controller with a few bytes of RAM.

Q4 – else–if Ladder vs switch

6 Marks · K4

Boundary test set (observed grades)

Mark	100	90	89	80	79	70	69	60	59	50	49	0	–1	101
Grade	S	S	A	A	B	B	C	C	D	D	F	F	?	?

Complete the Code – TODO 1

```
switch (marks / 10) {
    case 10:
    case 9:  return 'S';
    case 8:  return 'A';
    case 7:  return 'B';
    case 6:  return 'C';
    case 5:  return 'D';
    default: return 'F';
}
```

```
Output: 100:S 95:S 83:A 50:D 49:F -5:?
```

- **Ladder:** correct for all 14 inputs, but up to 5 comparisons; boundaries are easy to edit.
- **switch:** correct for all 14 because the range guard comes first; case labels must be single constants.
- **Ranges:** 75–84 cannot be written as a case; marks / 10 only works for boundaries that are multiples of 10.
- **Test set:** every boundary and its neighbour (90/89 … 50/49) plus 0, −1, 100, 101.

Q5 – Counting the Work of a Prime Loop

7 Marks · K4

Student-derived condition (principle → loop)

A divisor larger than √n implies one smaller than √n, so only i ≤ √n needs testing, i.e. i * i <= n (no sqrt() call). Stop with return 0 on the first divisor.

```
for (unsigned long i = 2; i * i <= n; i++) {
    checks++;
    if (n % i == 0) return 0;
}
return 1;             /* TODO 1 completed */
```

```
Output:
      97 prime     divisions = 8
     100 not prime divisions = 1
    7919 prime     divisions = 87
 1000003 prime     divisions = 999
```

Division counts (all versions)

n	i < n	for √n	while	do–while
97	95	8	8	8
100	1	1	1	1
7919	7917	87	87	87
1000003	1000001	999	999	999

Changing the condition to i < n gives the same primes but 1000× more work for the large prime (1,000,001 vs 999 divisions).

Expected output matches exactly.

Q5 – Inquiry & Sieve Alternative

7 Marks · K4

Boundary cases and overflow

- **do–while, n = 2, 3:** the body runs once before the test, so i = 2 divides n = 2 and wrongly reports "not prime". Guard with if (n < 4) return 1.
- **Overflow:** i * i <= n can wrap when i * i exceeds the type's maximum (int: i ≥ 46341). Safer: i <= n / i.
- **break / return:** leave the loop at the first divisor, which is why 100 needs 1 division.

Alternative: Sieve of Eratosthenes up to N

```
char flag[N + 1];  /* 1 = prime */
memset(flag, 1, N + 1); flag[0] = flag[1] = 0;
for (i = 2; i * i <= N; i++)
    if (flag[i])
        for (j = i * i; j <= N; j += i)
            flag[j] = 0;
/* query: flag[n]  -> O(1) */
```

Measure	Trial division	Sieve
Time	O(√n) per query	O(N log log N) once, O(1) per query
Space	O(1)	O(N) ≈ 1 MB for N = 1,000,003

Measured (n = 2…100,000): 2,745,694 divisions in total. The sieve pays off when many tickets are checked.

Q6 – Loan Schedule: Loop Design

8 Marks · K5

Hand-verified schedule: Rs 10,000 at 2%/month, Rs 2,000 repayment

Month	Opening	Interest (2%)	Repayment	Closing
1	10,000.00	200.00	2,000.00	8,200.00
2	8,200.00	164.00	2,000.00	6,364.00
3	6,364.00	127.28	2,000.00	4,491.28
4	4,491.28	89.83	2,000.00	2,581.11
5	2,581.11	51.62	2,000.00	632.73
6	632.73	12.65	645.38 (last)	0.00

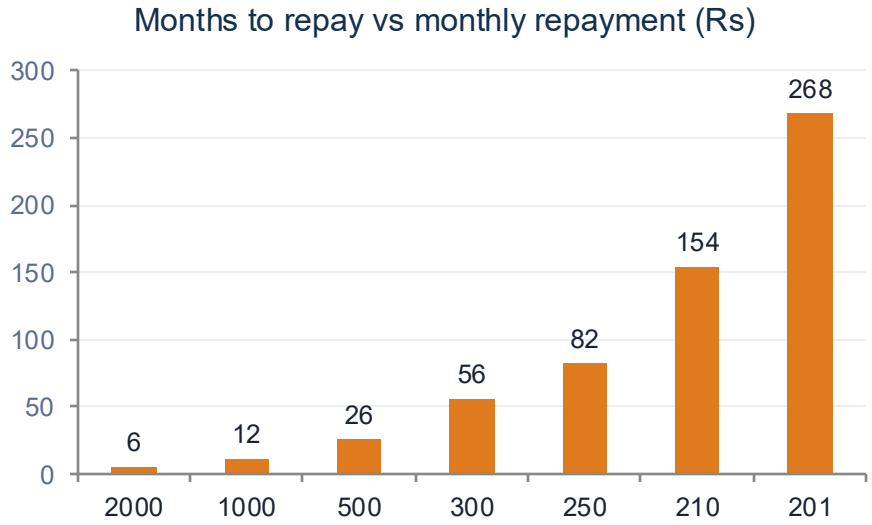
6 months, total interest Rs 645.38. Program output matches the hand calculation.

Design of the loop

- **Loop:** while – the loan may already be zero, and the count is unknown.
- **Condition:** balance > 0.005 && month < 360.
- **Why not != 0:** 10 × 0.1f = 1.000000119 ≠ 1.0, so it may never hit exactly 0.
- **Last month:** pay = min(E, balance + interest).
- **E ≤ interest:** balance never falls; infinite loop without the limit.

Q6 – Growth, Deployment & Formula

8 Marks · K5



Initial interest is Rs 200. As repayment falls towards 200 the months rise sharply; at 200 or 150 the loop hits the 360-month limit.

Deployment decision

```
if (E <= bal * r)
    puts("Repayment too low.");
while (bal > 0.005) {
    if (++month > 360) {
        puts("Not repaid in 360 mo.");
        break;
    }
    ...
}
```

Check up front (O(1)) and keep the 360-month break as a safety net: a clear message is better than a silent infinite loop.

Alternative: $n = -\log(1 - rP/E) / \log(1 + r)$

Formula: O(1) time and space, e.g. 5.32 → 6 months (matches); undefined when $rP/E \geq 1$ and loses precision near $E \approx rP$. Loop: O(n) time, but gives the exact last payment.

Q7 – Diagnose, Objective, Design, Predict

13 Marks · K5

Program chosen: Q5 prime check (trial division)

A. Diagnose (2)

All work is the divisions inside the loop. The count grows as √n: 999 for 1,000,003 and 2,745,694 over 2…100,000. Even divisors (4, 6, 8…) are tested although they cannot divide n once 2 has failed: repeated work.

B. Objective (2)

Cut divisions for prime inputs by about 50% (≤ 500 for n = 1,000,003) with identical results for every n.

C. Design (4)

Test 2 once, then try odd divisors only (i += 2). Property: an even i > 2 divides n only if 2 does. No extra memory, but a special case for n < 4.

D. Predict (1)

Divisions ≈ √n / 2: about 500 for 1,000,003, 44 for 7919, and about 1.4 million over 2…100,000 (a little less gain, since composites exit early).

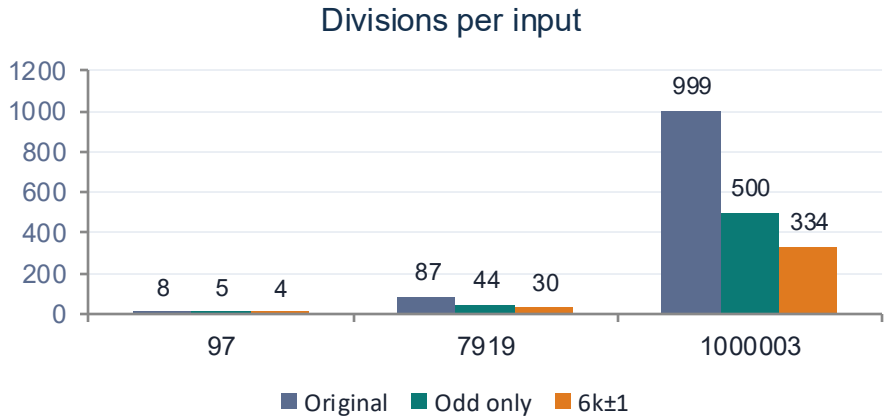
Q7 – Implement, Validate & Defend

13 Marks · K5

```
if (n < 2) return 0;  if (n < 4) return 1;
checks++; if (n % 2 == 0) return 0;
for (i = 3; i * i <= n; i += 2) {
    checks++;
    if (n % i == 0) return 0;
}
return 1;
```

n	Original	Odd only	6k±1
97	8	5	4
7919	87	44	30
1000003	999	500	334
Total 2…100000	2,745,694	1,445,439	1,031,092

Prediction met: 999 → 500 and 87 → 44. Over the range, divisions fall 47%.



F. Defend (2)

- Objective achieved: about 2× fewer divisions on primes, same O(√n), O(1) space.
- No gain for even n (1 division either way); 6k±1 saves more but is harder to read.
- The original is still fine for small n or when clarity matters most.

GitHub Submission Structure

```
CSA02-1965260110-Assignment2/
 ├─ README.md   (name, register no., gcc 13.3, compile command, run steps, result notes)
 ├─ Q1/  q1.c  output.png      ├─ Q5/  q5.c  output.png
 ├─ Q2/  q2.c  output.png      ├─ Q6/  q6.c  output.png
 ├─ Q3/  q3.c  output.png      └─ Q7/  q7.c  output.png
 └─ Q4/  q4.c  output.png
```

Compile: gcc -Wall -o qN qN.c (add -lm for Q6)
Commit at least once per task – progressive history, not a single upload.